Bertrand LECOEUR, H00483379

F70CF Continuous-time Finance- Assignment 2024

Question1

a)

```
set.seed(3379)
# Parameter configuration
T <- 4                  # Number of time units
n <- 500                # Number of time steps per unit of time
M <- 1000               # Number of paths
deltaT <- T / (T * n)       # Size of each time step

# Matrix to store Brownian Motion paths
W <- matrix(0, nrow = M, ncol = T * n + 1) # M paths, each with T*n + 1 points

# Path simulation
for (i in 1:M) {
        W[i, 2:(T * n + 1)] <- cumsum(sqrt(deltaT) * rnorm(T * n))
}
```

b)

```
# Extract values of W2, W3, and W4 for each path
W2 <- W[, 2 * n]       # W_t for t = 2
W3 <- W[, 3 * n]       # W_t for t = 3
W4 <- W[, 4 * n]       # W_t for t = 4

# (i) Calculation of the correlation Corr(W2, W4)
corr_W2_W4 <- cor(W2, W4)

# (ii) Calculation of the correlation Corr(W2, (W4 - W3)^3)
diff_W4_W3_cubed <- (W4 - W3)^3
corr_W2_diffW4W3_cubed <- cor(W2, diff_W4_W3_cubed)
```

Displaying results

cat("Corr(W2, W4) =", corr_W2_W4, "\n")

cat("Corr(W2, (W4 - W3)^3) =", corr_W2_diffW4W3_cubed, "\n")

```
> cat("Corr(W2, W4) =", corr_W2_W4, "\n")
Corr(W2, W4) = 0.7351132
> cat("Corr(W2, (W4 - W3)^3) =", corr_W2_diffW4W3_cubed, "\n")
Corr(W2, (W4 - W3)^3) = -0.01652154
```

c)

- Theoretical correlation of $Corr(W2, W4)$:

The Brownian motion Wt is a Gaussian process with independent increments and zero expectations, and for $s < t$:

$$\mathrm{Cov(Ws, Wt)} = \mathrm{s}$$

(Demonstrated in tutorial 2 Question 3).

Therefore, the correlation between $W2$ et $W4$ is calculated as:

$$\mathrm{Corr(W2, W4)} = \frac{\mathrm{Cov(W2, W4)}}{\sqrt{\mathrm{Var(W2)} \cdot \mathrm{Var(W4)}}}$$

Knowing that $\mathrm{Var(Wt)} = \mathrm{t}$ for a standard Brownian motion, we have:

$$\mathrm{Corr(W2, W4)} = \frac{2}{\sqrt{2 * 4}} = \frac{2}{2\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.7071$$

With question 1)b), we found a result of 0,735, which is almost equivalent to the calculated theoretical result.

- Theoretical correlation of $Corr(W2, (W4 - W3)^3)$:

As $W2$ and $W4 - W3$ are independent variables (because $W2$ is based on the interval $[0,2]$ and $W4 - W3$ on $[3,4]$), their covariance is zero:

$$Cov(W2, (W4 - W3)^3) = 0.$$

Since the covariance is zero, it follows that the correlation $Corr(W2, (W4 - W3)^3)$ is also zero:

$$Corr(W2, (W4 - W3)^3) = 0.$$

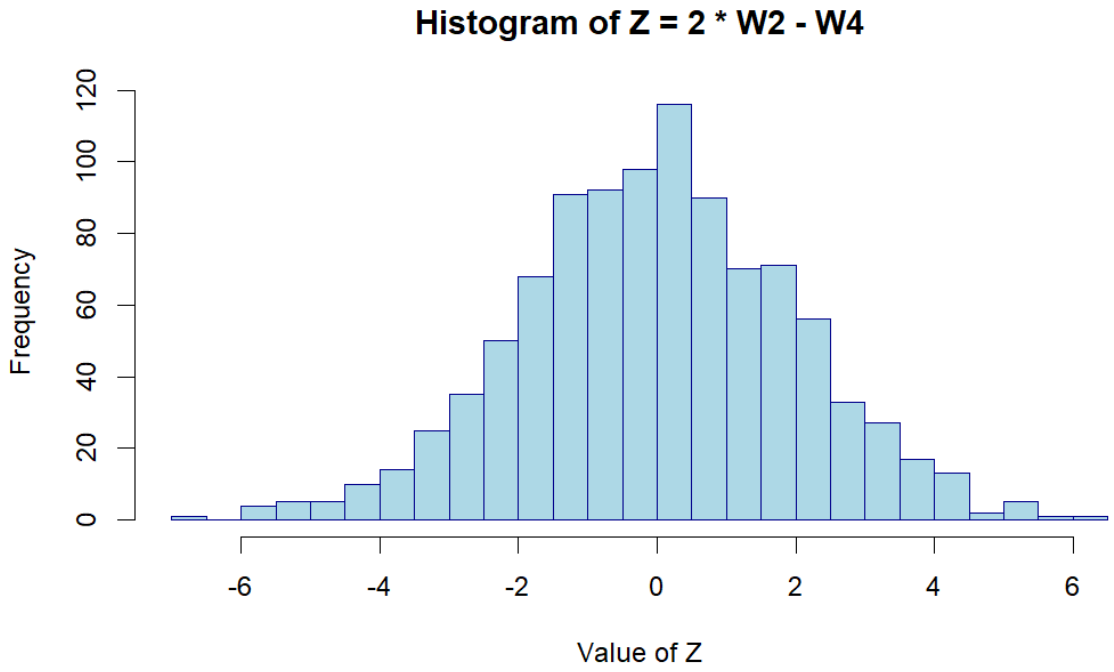
With question 1) b), we found a result of -0.017, which is almost equivalent to the calculated theoretical result.

d)

Calculation of the variable Z = 2 * W2 - W4 for each path

Z <- 2 * W2 - W4

```
# Plotting the histogram of Z
hist(Z, breaks = 30, col = "lightblue", main = "Histogram of Z = 2 * W2 - W4",
     xlab = "Value of Z", ylab = "Frequency", border = "darkblue")
```



e)

```
num_simulations <- 100       # Number of Monte Carlo simulations

# Initialize a vector to store the variance of Z for each simulation
variances_Z <- numeric(num_simulations)

# Monte Carlo simulation loop
for (sim in 1:num_simulations) {

        # Simulation of M paths of Brownian Motion
         W <- matrix(0, nrow = M, ncol = T * n + 1)
        for (i in 1:M) {
                W[i, 2:(T * n + 1)] <- cumsum(sqrt(deltaT) * rnorm(T * n))
```

```
    }

  # Extraction of W2 and W4 to calculate Z
  W2 <- W[, 2 * n]
  W4 <- W[, 4 * n]
  Z <- 2 * W2 - W4

  # Calculation of the variance of Z for this simulation
  variances_Z[sim] <- var(Z)
}

# Final estimation of the variance of Z by the average of variances from each simulation
variance_Z_MC <- mean(variances_Z)

# Displaying the result
cat("Monte Carlo estimation of the variance of Z:", variance_Z_MC, "\n")
```

```
> cat("Monte Carlo estimation of the variance of Z:", variance_Z_MC, "\n")
Monte Carlo estimation of the variance of Z: 3.990698
```

f)

Expectation of Z :

The Brownian motion Wt is centred, meaning that $E[Wt] = 0$ for all $t \geq 0$. So, the expectation of Z is:

$$E[Z] = E[2W2 - W4] = 2 \cdot E[W2] - E[W4] = 0.$$

Variance of Z :

The variance of Z is:

$$Var(Z) = Var(2W2 - W4)$$

Using the linearity property of the variance for Gaussian random variables (and the covariance of Brownian motion), we have:

$$Var(Z) = 4 \cdot Var(W2) + Var(W4) - 2 \cdot Cov(2W2, W4)$$

Knowing that $Var(Wt) = t$ for a Brownian motion, we obtain:

$$Var(Z) = 4 \cdot 2 + 4 - 2 \cdot 2 \cdot Cov(W2, W4)$$

We use here that $Cov(W2, W4) = 2$, which gives:

$$Var(Z) = 8 + 4 - 8 = 4.$$

Now, because Z is a linear combination of normal variables (Brownian motion is Gaussian), Z also follows a normal law. We found that:

- $E[Z] = 0$,
- $Var(Z) = 4$.

Thus, Z follows a normal distribution of parameter $N(0,4)$, so $Z \sim N(0,4)$.

So, in question 1) e), we estimated the variance of Z using Monte Carlo simulations. These estimates give us a result very close to 4, with a slight variation due to sampling error.

Question2

a)

```
Delta_t <- 1/n  # Time step size

Z_0_a <- 0.25     # Initial value of Z

W_0_a <- 0        # Initial value of W


# Time sequence

t <- Delta_t * (0:(T * n))  # Time sequence for the plot


# Initialization of vectors for Z and W

Z_a <- numeric(T * n + 1)  # Storage vector for Z

W_a <- numeric(T * n + 1)  # Storage vector for W

Z_a[1] <- Z_0_a             # Initial condition for Z

W_a[1] <- W_0_a             # Initial condition for W


# Simulation of W and Z

for (k in 1:(T * n)) {

        # Increment of W with a standard Brownian motion

        dW_a <- rnorm(1, mean = 0, sd = sqrt(Delta_t))

        W_a[k + 1] <- W_a[k] + dW_a  # Update of W


       # Update of Z

       Z_a[k + 1] <- Z_a[k] + (-1/2 * sin(W_a[k])) * Delta_t + cos(W_a[k]) * dW_a
```

```
}
```

b)

```
# Simulation parameters
Z_0_b <- 0.25          # Initial value of Z

# Initialization of vectors for Z and W
Z_b <- numeric(T * n + 1)     # Storage vector for Z
Z_b[1] <- Z_0_b               # Initial condition for Z
W_b <- numeric(T * n + 1)     # Storage vector for W
W_b[1] <- 0                   # Initial condition for W

# More precise simulation of W and Z
Z_increments <- rnorm(T * n)  # iid increments of N(0,1) to simulate W
for (k in 1:(T * n)) {
        # Increment of W with a standard Brownian motion
        dW_b <- sqrt(Delta_t) * Z_increments[k]
        W_b[k + 1] <- W_b[k] + dW_b

        # Update of Z with an exponential discretization
        Z_b[k + 1] <- Z_b[k] * exp((-1/2 * sin(W_b[k])) * Delta_t + cos(W_b[k]) * dW_b)
}
```

First, in the loop, we update Z using an exponential formula. This reduces simulation errors compared to the method used in question 1) a).

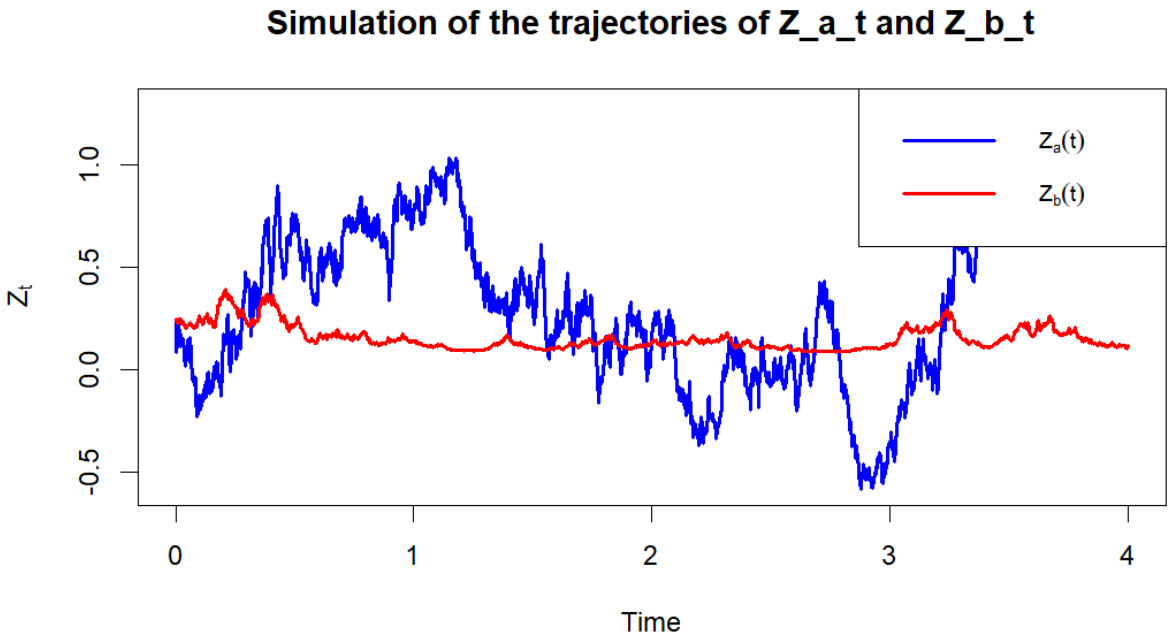
Next, we use a sequence of iid values of $N(0,1)$ to generate the increments of Wt. Thus, this exponential method is more stable for SDEs having diffusion derivatives which depend on the Brownian variable (here W).

c)

```
# Plotting the trajectory of Z_a
plot(t, Z_a, type = "l", col = "blue", lwd = 2,
    xlab = "Time", ylab = expression(Z[t]), main = "Simulation of the trajectories of Z_a_t and Z_b_t")
```

```
# Adding the trajectory of Z_b on the same plot
lines(t, Z_b, col = "red", lwd = 2)

# Adding a legend to differentiate the trajectories
legend("topright", legend = c(expression(Z[a](t)), expression(Z[b](t))),
       col = c("blue", "red"), lwd = 2, cex = 0.8)
```



d)

In question 2) a), the additive method can lead to cumulative errors, especially for nonlinear equations. This results in a more approximate trajectory of Zt that may drift over time.

In question 2) b), the exponential method is better suited for multiplicative processes and limits error accumulation by adjusting proportionally to the current value. This keeps the Zt trajectory closer to the expected theoretical values.

Thus, the exponential method provides a trajectory of Zt which remains closer to the theoretically expected values, while the method in question 2) a) tends to deviate over time. Indeed, the exponential approach is more accurate for equations like this, where the changes are influenced by the current level of the process Z and W.

Question 3 :

Having this code at the beginning:

```
# F70CF 2024/25

# Assignment - Questions 3 and 4
```

```
#####################################################
####  REPLACE 1234 IN THE FOLLOWING LINE WITH ####
####  THE LAST FOUR DIGITS OF YOUR STUDENT ID ####
set.seed(3379)
#####################################################
rm(list = ls())    # delete everything from the workspace
graphics.off()     # close all existing graphics/plots
source("F70CF.R")  # load functions for implicit FD
#### some constants ############################
K = 2            # strike price
T = 4
r = 0.05         # interest rate p.a.
sigma = 0.4 + rexp(1, 10)      # volatility p.a.
S_max = 8        # max share price for FD
deltaS = 0.05
M = S_max / deltaS  # number of share price intervals for FD
deltaT = 1 / 120     # length of a time step in FD method
N = T / deltaT      # number of time intervals for FD
#####################################################
a)
# Black-Scholes Formula for European Put Option

# Function to calculate the price of a European put option
black_scholes_put <- function(S, K, T, r, sigma) {
        # S: current stock price
        # K: strike price
        # T: time to maturity in years
        # r: risk-free interest rate
        # sigma: volatility of the underlying asset
```

```
    # Calculation of d1 and d2
    d1 <- (log(S / K) + (r + sigma^2 / 2) * T) / (sigma * sqrt(T))
    d2 <- d1 - sigma * sqrt(T)

    # Calculation of the put option price
    put_price <- K * exp(-r * T) * pnorm(-d2) - S * pnorm(-d1)

    return(put_price)
}
```

Variables d1 and d2 respectively represent the adjusted probabilities of the stock price relative to the strike price, considering the risk-free rate and volatility.

To calculate the sale price, we use the difference between the present value of the exercise of the option (K * exp (-r * T) * pnorm(-d2)) and the current product of the price of the asset and the probability of non-exercise (S * pnorm(-d1)).

b)

The Strap payoff is given by :

$$X(ST) = \begin{cases} 2(K - ST), & if\ ST < K \\ 3(ST - K), & if\ ST \geq K \end{cases}$$

, where K is the strike price (here K=2).

To reproduce this payment structure with European options, it is possible to use a combination of 2 puts and 3 calls, all with the same K strike price and maturity T.

1. If $ST < K$:

   The payoff of the 2 put options is: $2(K - ST)$.

   The call options expire worthless in this case, so their contribution is 0.

   The total payoff in this case is therefore $2(K - ST)$, which corresponds to the Strap payoff when ST<K.

2. If $ST \geq K$:

   Put options expire worthless, so their contribution is 0.

   The payoff for the 3 call options is: $3(ST - K)$.

   The total payoff in this case is therefore $3(ST - K)$, which also corresponds to the Strap payoff when $ST \geq K$.

We can therefore replicate the Strap payoff with the following combination of options:

$$Strap = 2 \times Put + 3 \times Call$$

where each put and call option has a strike price K and a maturity T.

c)

```
# Black-Scholes Function for a European call option
black_scholes_call <- function(S0, K, T, sigma, r) {
        # S0: Current stock price
        # K: Strike price
        # T: Maturity (4 years)
        # sigma: Annual volatility
        # r: Risk-free rate

        d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T) / (sigma * sqrt(T))
        d2 <- d1 - sigma * sqrt(T)
        call_price <- S0 * pnorm(d1) - K * exp(-r * T) * pnorm(d2)
        return(call_price)
}

# Black-Scholes Function for a European put option
black_scholes_put <- function(S0, K, T, sigma, r) {
        # Same parameters as black_scholes_call

        d1 <- (log(S0 / K) + (r + 0.5 * sigma^2) * T) / (sigma * sqrt(T))
        d2 <- d1 - sigma * sqrt(T)
        put_price <- K * exp(-r * T) * pnorm(-d2) - S0 * pnorm(-d1)
        return(put_price)
}

# Calculation of Strap price
strap_price <- function(S0, K, T, sigma, r) {
        # Price of 2 put options and 3 call options
        put_price <- black_scholes_put(S0, K, T, sigma, r)
```

```
    call_price <- black_scholes_call(S0, K, T, sigma, r)


    strap_price <- 2 * put_price + 3 * call_price
    return(strap_price)
}
```

The “black_scholes_call” and “black_scholes_put” functions calculate the prices of European call and put options respectively using the Black-Scholes formula. These functions take the necessary basic parameters (initial stock price S0, strike price K, maturity T, sigma volatility and risk-free rate r).

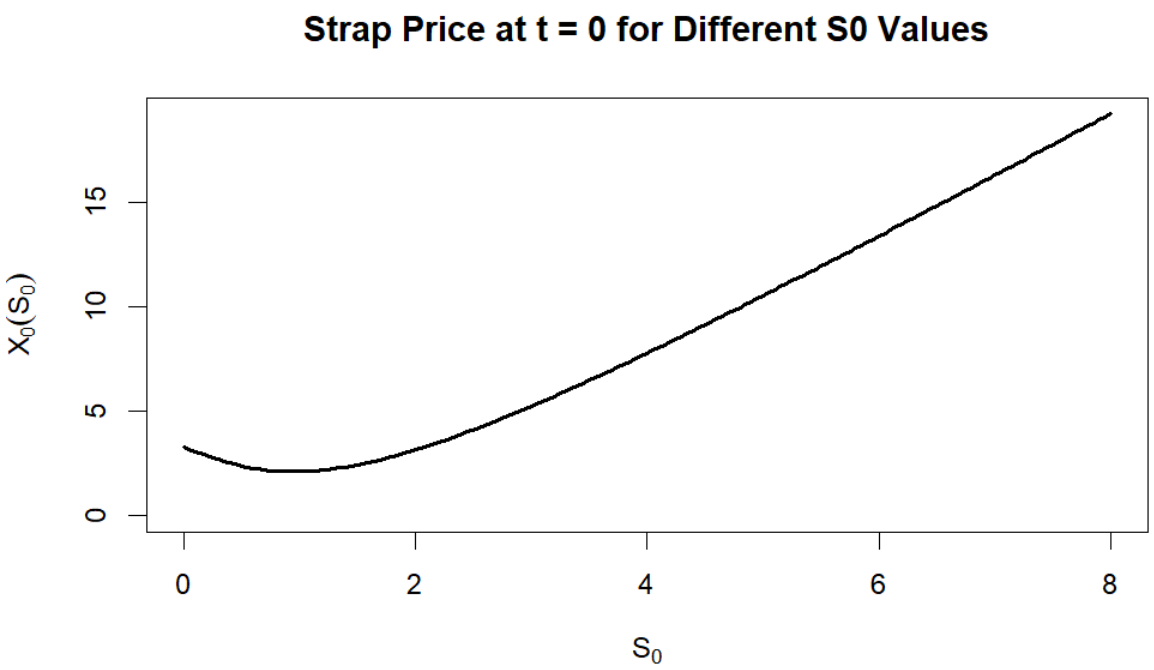
The “strap_price” function allows to calculate the price of the Strap. It uses Black-Scholes functions for calls and puts, then combines them to calculate the total price of a Strap by multiplying by the number of options in each component of the Strap (two puts and three purchase options as found in question b)).

d)

```
# Vector of S0 values
S0_values <- seq(0, S_max, by = deltaS)


# Calculation of Strap prices for each S0 value
BS_x0 <- sapply(S0_values, strap_price, K = K, T = T, sigma = sigma, r = r)


# Plotting the graph
plot(S0_values, BS_x0, type = "l", col = "black", lwd = 2,
    xlab = expression(S[0]), ylab = expression(X[0](S[0])),
    main = "Strap Price at t = 0 for Different S0 Values",
    ylim = c(0, max(BS_x0)))
```



Question 4 :

a)

```
# Definition of the stock price range and initialization of the option value matrix

stock_prices <- seq(0, S_max, by = deltaS)  # Generating stock prices between 0 and S_max

option_matrix <- matrix(0, nrow = length(stock_prices), ncol = N + 1)  # Initializing the matrix for option values


# Initialization of the final payoff for the Strap at maturity time

option_matrix[, N + 1] <- 3 * pmax(stock_prices - K, 0) + 2 * pmax(K - stock_prices, 0)


# Definition of boundary conditions: stock price at 0 and at S_max

limit_low <- 2 * K * exp(-r * (T - (0:(N)) * deltaT))  # Pre-computation of the limit for S = 0 at each time step

option_matrix[1, ] <- limit_low  # Applying the lower boundary condition on the first row


limit_high <- rep(3 * S_max - K, N + 1)  # Pre-computation of the limit for S = S_max

option_matrix[length(stock_prices), ] <- limit_high  # Applying the upper boundary condition on the last row
```

Applying the finite difference method to calculate the price of the European Strap

Choosing the implicit method with the argument explicit = FALSE

option_matrix <- FD_Euro(option_matrix, r, sigma, deltaT, explicit = FALSE)

The code begins by setting up a grid of stock prices ranging from 0 to S_max=8 with steps of deltaS, as defined in F70CF_assignment_2024.R. This creates a stock_prices vector that covers possible prices for the Strap option. The option_matrix is initialized as a zero matrix to store option prices at each combination of stock price and time step.

The final payoff at maturity T for the Strap is $3max(S-K,0)+2max(K-S,0)$, which gives three times the payoff of a call and twice that of a put, depending on the final stock price.

For boundary conditions, at $S=0$, the value is approximately $2K$, representing the put payoff discounted to each time step. This is stored in the first row of option_matrix. At $S=S\_max$, where the stock price is high, the payoff reflects the call component, approximated as $3Smax-K$, and stored in the last row.

The finite difference method is applied using FD_Euro with an implicit time-stepping scheme (explicit = FALSE). This approach calculates option prices by iteratively working backward from the fianl payoff, adjusting for volatility and interest rate. Boundary conditions ensure correct values across the price grid, where $S=0$ reflects the put component and $S=S\_max$ reflects the call component.

b)

Vector of S0 values

S0_values <- seq(0, S_max, by = deltaS)

Calculation of Strap prices for each S0 value

BS_x0 <- sapply(S0_values, strap_price, K = K, T = T, sigma = sigma, r = r)

Initial plot of the graph

plot(S0_values, BS_x0, type = "l", col = "black", lwd = 2,

    xlab = expression(S[0]), ylab = expression(X[0](S[0])),

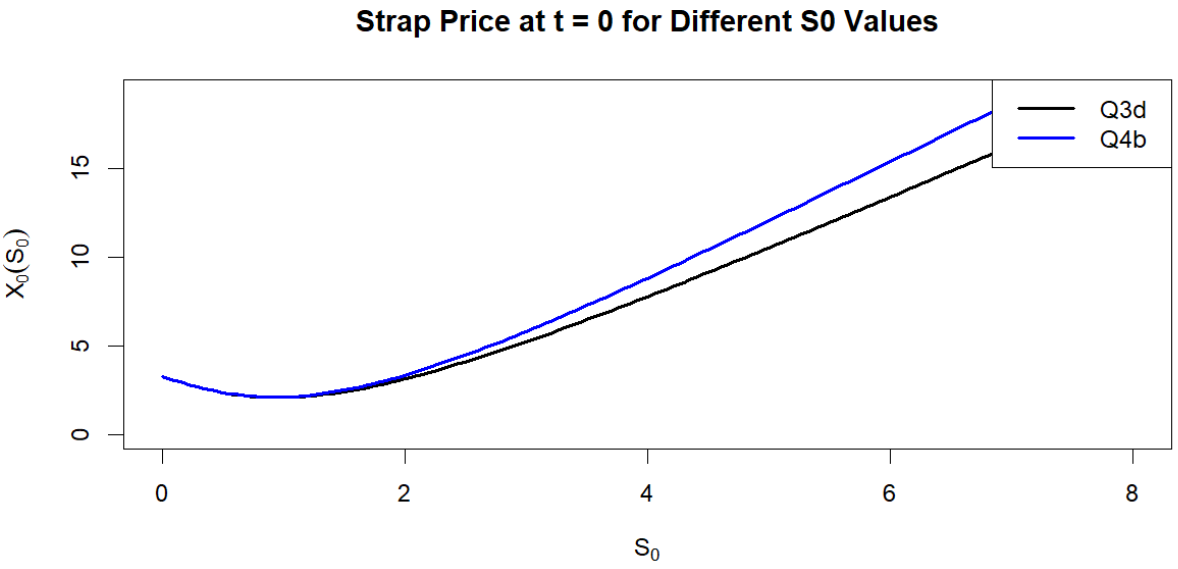
    main = "Strap Price at t = 0 for Different S0 Values",

    ylim = c(0, max(BS_x0)))

Adding the prices for t = 0 obtained in Question 4a

lines(S0_values, option_matrix[, 1], col = "blue", lwd = 2) # Replace `option_matrix[, 1]` with prices obtained in Question 4a

legend("topright", legend = c("Q3d", "Q4b"), col = c("black", "blue"), lwd = 2)



c)

The two curves are extremely close, which is expected because they both represent estimates of the same option price. The curve in Question 3) d was calculated using a specific method (the Black-Scholes formula), while the curve added in Question 4b is based on an approximation by the finite difference method.

The very small difference between the two curves underlines the effectiveness of the finite difference method for estimating option prices, and validates its use for the Strap.